Tutorial 05 — Instructor notes

ENVX2001 – Applied Statistical Methods

Semester 1

Contents

Student document: Tutorial 05

Before you start

Sort these out before students arrive:

- RStudio open, fresh session. All data is simulated in the tutorial code, so no files to download.
- `ggplot2` needs to be installed. Nothing else.
- Count how many students are present. You need this for Exercise 1.
- Exercise 1 is a class activity on randomisation (~15 min). Exercises 2 and 3 walk through CRD and RCBD analyses (~10 min and ~15 min). Budget about 40 minutes total.

Exercise 1: Testing randomisation

This is a live class activity. You survey the whole class first (non-random), then randomly pick a subset and repeat.

Part A — Non-random sample

What to do

Ask every student: "Do you know the student sitting next to you? Yes or No." Hands up for Yes, down for No. Count them.

Update the code with the real numbers:

```
CODE
total_nonrandom ← 60 # replace with actual class size
count_yes_nonrandom ← 38 # replace with actual Yes count
count_no_nonrandom  ← 22 # replace with actual No count

class_size ← total_nonrandom
```

What to say

This is a convenience sample. We asked everyone, but the question itself introduces bias because students who sit next to friends will answer differently from those who ended up in a random seat. Friends cluster. That is the whole point.

Part B — Percentages

What to do

Run the percentage calculation live.

```
CODE
prop_yes_nonrandom ← (count_yes_nonrandom / total_nonrandom) * 100
prop_no_nonrandom  ← (count_no_nonrandom  / total_nonrandom) * 100

cat("Non-random sample % Yes:", round(prop_yes_nonrandom, 1), "%\n")
cat("Non-random sample % No :", round(prop_no_nonrandom, 1),  "%\n\n")
```

What to say

The Yes percentage is usually high, 60-70%, because friends sit together. That is the bias you are trying to demonstrate.

Part C — Random sample

What to do

Run the random selection code to pick a subset. Call out the selected student numbers. Ask only those students again, record the counts.

```
CODE
set.seed(123)
student_ids ← seq_len(class_size)
rand_num ← runif(class_size)
class_df ← data.frame(id = student_ids, rand_num = rand_num)

random_sample_size ← 30

set.seed(202)
sampled_ids ← sample(class_df$id, size = random_sample_size, replace = FALSE)
sampled_ids
```

Update the random sample counts and run the comparison:

```
CODE
count_yes_random ← 14 # replace with actual count
count_no_random  ← 16 # replace with actual count

prop_yes_random ← (count_yes_random / random_sample_size) * 100
prop_no_random  ← (count_no_random  / random_sample_size) * 100

cat("Random sample % Yes:", round(prop_yes_random, 1), "%\n")
cat("Random sample % No :", round(prop_no_random, 1),  "%\n\n")
```

Then the bias and bar plot:

```
CODE
bias_yes ← prop_yes_nonrandom - prop_yes_random
bias_no  ← prop_no_nonrandom  - prop_no_random

cat("Estimated bias in % Yes (non-random - random):", round(bias_yes, 1), "%\n")
cat("Estimated bias in % No  (non-random - random):", round(bias_no, 1),  "%\n\n")
```

```
CODE
percent_df ← data.frame(
  sample_type = rep(c("Non-random", "Random"), each = 2),
  response    = rep(c("Yes", "No"), times = 2),
  percent     = c(prop_yes_nonrandom, prop_no_nonrandom, prop_yes_random, prop_no_random)
)

library(ggplot2)

ggplot(percent_df, aes(x = response, y = percent, fill = sample_type)) +
  geom_col(position = "dodge") +
  labs(x = "Response", y = "Percent", fill = "Sample type")
```

What to say

The random sample usually has a lower Yes percentage. The difference is our estimate of bias.

If the percentages happen to come out similar, that is fine. It does not mean randomisation failed. It means seating was not very clustered in this class. The principle still holds.

The thing to land here: convenience sampling can systematically over- or under-represent certain groups. Random sampling gives every unit an equal chance, which cuts out that systematic bias. This is exactly why we randomise in experiments too. If you do not randomly assign treatments, systematic differences between groups can confound your results.

Common issues

> ⚠ Warning
>
> **Students do not know their number.** If you forgot to assign numbers at the start, just number by row and seat ("row 3, seat 5 is student 17"). Or skip the physical selection and use the pre-filled example data.

> ⚠ Warning
>
> **The bias is small or goes the wrong way.** This happens with smaller classes or when seating is genuinely mixed. Use it as a teaching moment: randomisation does not guarantee a perfect result every single time, but on average it produces less biased estimates.

Exercise 2: CRD

You lead this one end to end. Students follow along and run each code block.

What to do

Show the CRD diagram (Slide1.PNG), then work through the simulated experiment:

1. Show the statistical model and explain each term.
2. Expand the code-folded blocks for treatment assignment and data simulation. Run them.
3. Before showing the ANOVA output, ask: "Do you think there is a significant difference?"
4. Run the ANOVA and read the output together.

```
CODE
set.seed(123)
treatments ← rep(c("A", "B", "C"), each = 4)
pots ← data.frame(pot_id = 1:12, treatment = sample(treatments))

set.seed(456)
plant_heights ← data.frame(
  pot_id = rep(1:12, each = 5),
  plant_id = 1:60,
  height = rnorm(60, mean = rep(c(10, 15, 20), each = 20), sd = 2)
)
plant_heights ← merge(plant_heights, pots, by = "pot_id")

anova_result ← aov(height ~ treatment, data = plant_heights)
summary(anova_result)
```

What to say

Walk through $Y_{ij} = \mu + \alpha_i + \epsilon_{ij}$ in plain language. The "word version" (plant height = overall mean + fertilizer treatment + random error) is there for students who find the notation intimidating. Spend a moment on it.

We are generating fake data where treatment A has a true mean of 10, B has 15, and C has 20. The ANOVA will easily detect these differences. That is deliberate. The point is to practise reading the output, not to wrestle with an ambiguous result.

When you get to the ANOVA table: a large F means the between-group differences are much bigger than the within-group noise. That is all F measures.

Worth mentioning: each pot is the experimental unit (the treatment is applied to the pot), and each plant is a measurement unit. With 5 plants per pot there are 12 experimental units but 60 observations. In a real analysis you would need to account for this, but the simulation ignores the nesting to keep things simple.

Common issues

> ⚠ Warning
>
> **"Why are there 5 plants per pot but we are not averaging them?"** Fair question. In a real experiment with sub-sampling you would either average within pots or use a nested model. This simulation treats each plant as independent to keep the ANOVA straightforward. Acknowledge the shortcut and mention that more realistic designs come up later in the course.

Exercise 3: RCBD

Same as Exercise 2: you lead, students follow.

What to do

Show the RCBD diagram (Slide2.PNG), then work through the simulated experiment:

1. Show the statistical model. Point out the extra β_j (block) term compared to the CRD model.
2. Run the block assignment, data simulation, and ANOVA code.
3. Walk through the percentage of variation explained calculation.
4. Before showing the R code for the SS calculation, ask students to try it by hand from the ANOVA table.

```
CODE
set.seed(123)
blocks ← rep(c("North", "South", "East", "West"), each = 3)
treatments ← rep(c("A", "B", "C"), times = 4)
pots ← data.frame(pot_id = 1:12, block = blocks, treatment = sample(treatments))

set.seed(456)
plant_heights ← data.frame(
  pot_id = rep(1:12, each = 5),
  plant_id = 1:60,
  height = rnorm(60, mean = rep(c(10, 15, 20), each = 20) + rep(c(2, -2, 1, -1), each = 15), sd =
2)
)
plant_heights ← merge(plant_heights, pots, by = "pot_id")

anova_result ← aov(height ~ block + treatment, data = plant_heights)
summary(anova_result)
```

For the variance partitioning:

```
CODE
anova_table ← summary(anova_result)[[1]]
ss_total ← sum(anova_table$`Sum Sq`)
ss_treatment ← anova_table$`Sum Sq`[2]
ss_block ← anova_table$`Sum Sq`[1]
percent_treatment ← (ss_treatment / ss_total) * 100
percent_block ← (ss_block / ss_total) * 100
cat("Percentage of variation explained by treatments:", round(percent_treatment, 1), "%\n")
cat("Percentage of variation explained by blocks:", round(percent_block, 1), "%\n")
```

What to say

The only difference in R between a CRD and an RCBD is adding `block` to the formula before `treatment`. The block term absorbs variation that would otherwise inflate the residual, making treatment effects easier to detect.

On formula order: `aov(height ~ block + treatment)` tests block first, then treatment. In a balanced design the order does not change the treatment F-test. Mention it briefly but do not go deep. It matters for unbalanced designs, which come later.

For the percentage of variation explained, walk through one calculation by hand from the ANOVA table before showing the R code. It is SS for a source divided by total SS, times 100. Emphasise that students need to be able to do this with a calculator in assessments.

Look at the block p-value with students. If it is significant, blocking explained real variation that would have inflated the residual in a CRD. If it is not significant, blocking was unnecessary (but did no harm).

Finish by connecting to the lab: this week students will analyse real CRD and RCBD data (dingo/fox camera traps and fire/mammal abundance) using the same `aov()` and variance partitioning workflow. The tutorial gives them practice on simulated data so the lab is not their first time.

Common issues

> ⚠ Warning
>
> **Students confuse the block and treatment rows in the ANOVA table.** The row order matches the formula: block first, treatment second. Point to the row names explicitly.

> ⚠ Warning
>
> **"Why do we not test interactions between block and treatment?"** In an RCBD we assume no block-by-treatment interaction. If the interaction were real, we would need more replicates per cell to estimate it. Factorial designs come later in the course.

⚠ Warning

Hand calculation errors with SS. Students sometimes grab the wrong rows or forget to sum all three SS for the total. Walk through it slowly: Total SS = Block SS + Treatment SS + Residual SS. Then percentage = (source SS / Total SS) × 100.

Wrapping up

What to say

Randomisation reduces bias in both sampling and experiments. A CRD randomly assigns treatments to homogeneous units. An RCBD adds blocks to account for known variation, which shrinks the residual and makes treatment effects easier to pick up. Those are the ideas from today.

The exam questions at the end of the tutorial are for self-study. Do not walk through them in class, but encourage students to have a go before the lab.

In the lab, students work with real data (camera trap surveys and fire ecology) using the same `aov()` and variance partitioning workflow they practised here.